

EXPERIMENT – 9

Name: S. Mahalakshmi

Roll No.: 240701301

AIM:

To design input forms that validate user data such as email and phone number and display error messages using HTML, CSS, and JavaScript with Validator.js.

Setting Up the HTML Form:

Program:

```
<!DOCTYPE html>

<html>

<head>

    <title>Form Validation</title>

</head>

<body>

<h2>Registration Form</h2>

<form>

    <label>Email:</label><br>

    <input type="text"><br><br>

    <label>Phone Number:</label><br>

    <input type="text"><br><br>

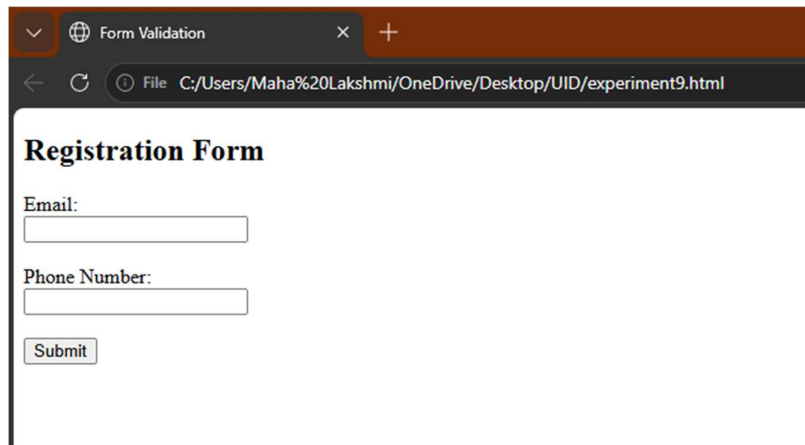
    <button type="submit">Submit</button>

</form>

</body>

</html>
```

Output:

A screenshot of a web browser window. The title bar shows 'Form Validation' with a close button. The address bar shows the file path 'C:/Users/Maha%20Lakshmi/OneDrive/Desktop/UID/experiment9.html'. The main content area displays a 'Registration Form' with two input fields: 'Email:' and 'Phone Number:'. Below these fields is a 'Submit' button. The form is styled with a white background and a thin border.

The basic HTML form is displayed on the screen. It contains input fields for entering the email address and phone number, along with a submit button. No validation or styling is applied at this stage.

In this step, a simple form structure is created using HTML. The form includes input fields for user data collection and serves as the foundation for further styling and validation.

Styling the Form using CSS:

Program:

```
<style>
```

```
body {
```

```
    font-family: Arial;
```

```
    background-color: #f2f2f2;
```

```
}
```

```
form {
```

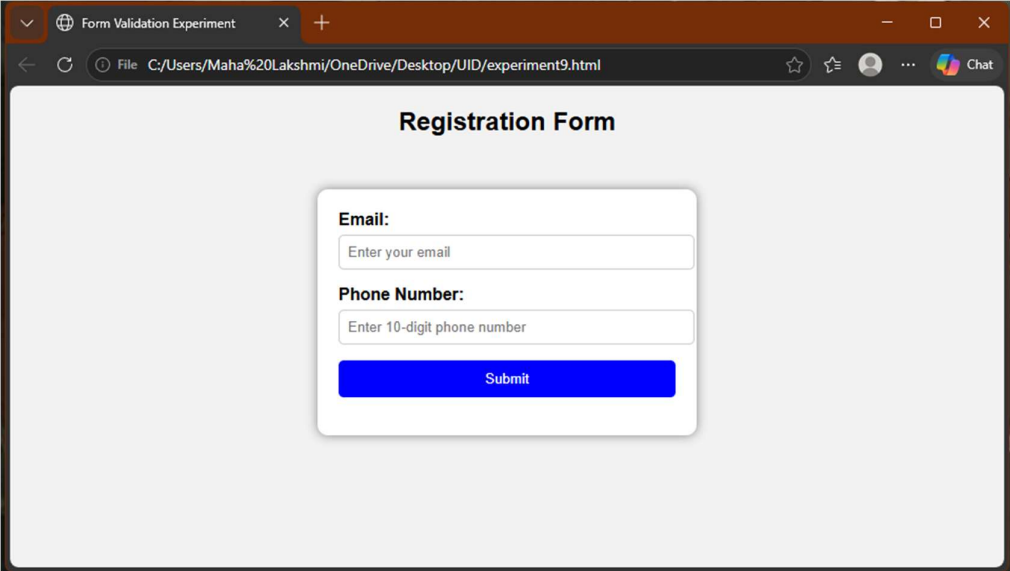
```
    background: white;
```

```
    padding: 20px;
```

```
    width: 300px;
```

```
margin: auto;
border-radius: 10px;
}
input {
width: 100%;
padding: 8px;
}
button {
background-color: blue;
color: white;
padding: 10px;
border: none;
width: 100%;
}
</style>
```

Output:



The screenshot shows a web browser window with the title "Form Validation Experiment". The address bar displays the file path "C:/Users/Maha%20Lakshmi/OneDrive/Desktop/UID/experiment9.html". The main content area features a "Registration Form" centered on a light gray background. The form is a white box with rounded corners and a subtle shadow. It contains two input fields: "Email:" with the placeholder text "Enter your email" and "Phone Number:" with the placeholder text "Enter 10-digit phone number". Below these fields is a blue "Submit" button with white text.

The form is displayed with improved visual appearance. It is centered on the screen with a styled layout, including proper spacing, background color, and a formatted submit button.

CSS is applied to enhance the appearance of the form. Styling improves readability and user experience by organizing the layout and making the interface visually appealing.

Adding JavaScript Validation using Validator.js:

For Validator.js:

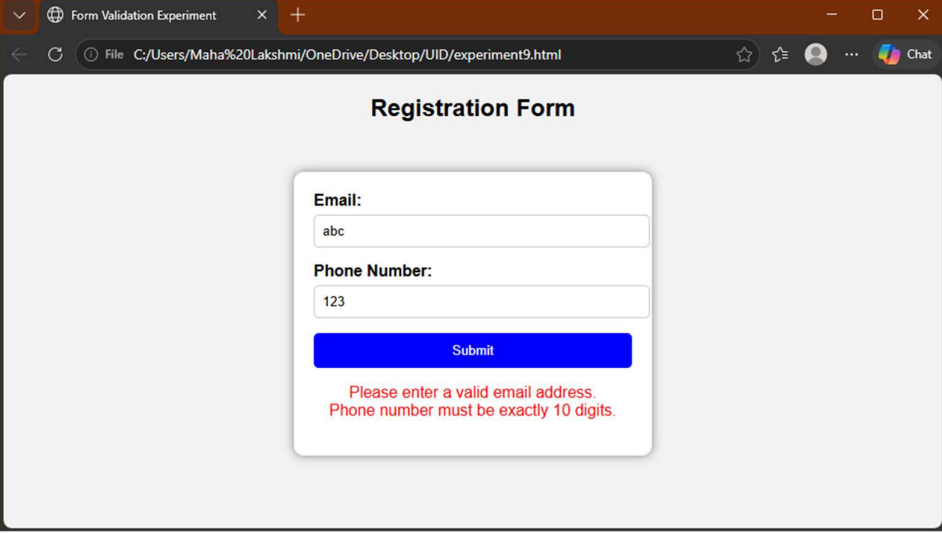
```
<script  
src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.7.0/validator.min.js"></  
script>
```

```
<script>
```

```
document.querySelector("form").addEventListener("submit", function(event) {  
    let email = document.querySelectorAll("input")[0].value.trim();  
    let phone = document.querySelectorAll("input")[1].value.trim();  
    let error = "";  
    if (!validator.isEmail(email)) {  
        error += "Invalid email format.<br>";  
    }  
    if (!validator.isMobilePhone(phone, 'en-IN')) {  
        error += "Invalid phone number.<br>";  
    }  
    if (error !== "") {  
        event.preventDefault();  
        document.body.innerHTML += "<p style='color:red'>" + error + "</p>";  
    }  
}
```

```
});  
</script>
```

Output:



The screenshot shows a web browser window titled 'Form Validation Experiment'. The address bar shows the file path 'C:/Users/Maha%20Lakshmi/OneDrive/Desktop/UID/experiment9.html'. The main content area displays a 'Registration Form' with two input fields: 'Email:' containing 'abc' and 'Phone Number:' containing '123'. Below the fields is a blue 'Submit' button. Red error messages are displayed below the button: 'Please enter a valid email address.' and 'Phone number must be exactly 10 digits.'

When invalid data is entered, appropriate error messages are displayed on the screen. Incorrect email format or invalid phone number triggers validation errors. When valid data is entered, the form is submitted successfully.

JavaScript along with Validator.js is used to validate the input fields. The `isEmail()` and `isMobilePhone()` functions ensure that the entered data follows the correct format. Error messages are shown for invalid inputs.

RESULT:

The input form was successfully created and validated using HTML, CSS, and JavaScript. The Validator.js library was used to check the correctness of email and phone number inputs, and error messages were displayed for invalid data.

CONCLUSION:

This experiment demonstrates how form validation improves data accuracy and user experience. Using Validator.js simplifies the validation process and ensures reliable input handling.